COM3021/COMM115 Data Science at Scale

Distributed Architectures - Partitioning

DR HUGO BARBOSA

Distributing data across multiple nodes

- Replication

  Keeping a copy of the data in each node

- Partitioning

  Different nodes will store subsets of the data called partitions

Partitioning

Introduction

- For very large datasets, having exact copies of the data on different nodes is not sufficient
- A partitioned database stores different subsets of the data in different nodes
- Partitions Different names in different DBMS:

  Shard: MongoD and Firestore

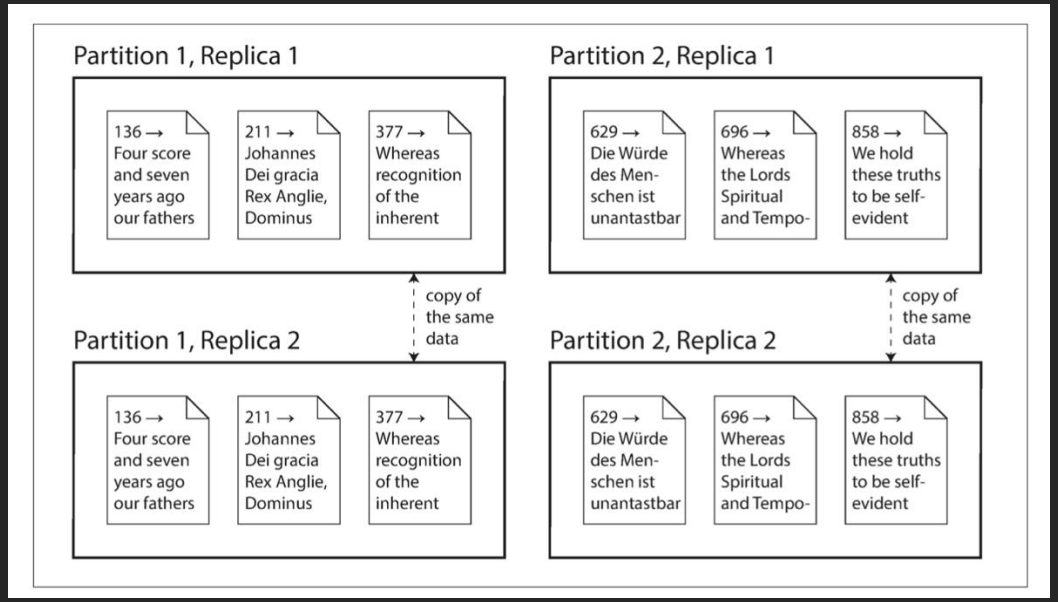
  Region: Hbase

  Vnode: Cassandra

Partitioning

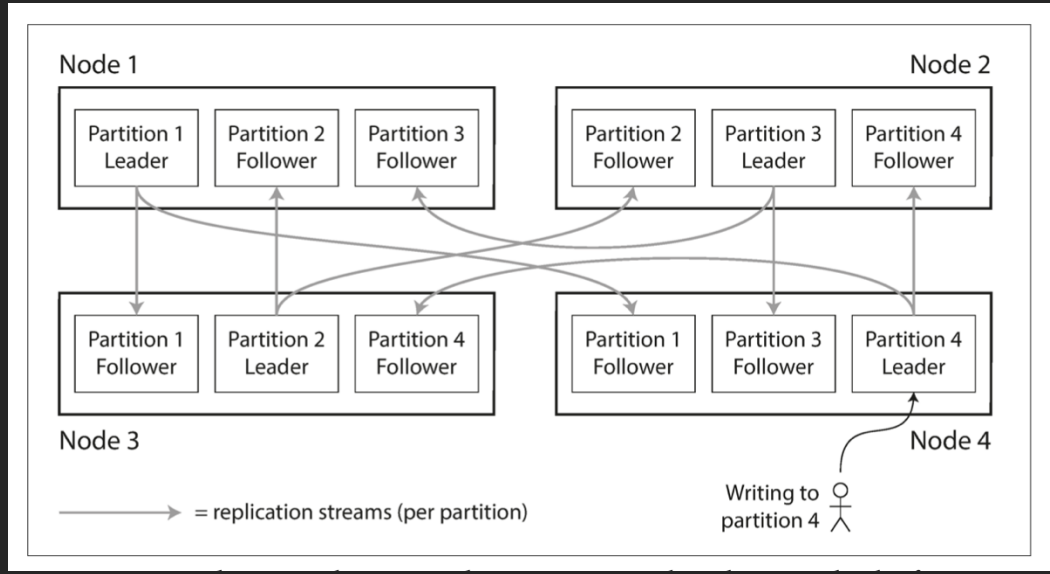
- Each node stores a ***subset*** of the data (i.e., partition)
- A node can store more than one partition
- Each piece of data will belong to **exactly one partition**
- Focus on *scalability*

  The query load is spread across nodes
- Often times in combination with replication



Partitioning

- In a Leader-Followers replication architecture, each partition's leader is assigned to one node while the followers are assigned to other nodes



Partitioning

- The goal is to spread the data and the query load **evenly** across nodes.
- If the partitioning is fair, 10 nodes will handle 10x as much load and data*.

  *Ignoring replication and overhead

  Otherwise we might end up with skewed partitioning in which some nodes will handle a disproportionally larger amount of data and load.

  - Hot spot nodes

    e.g., some parts of the data are more requested then others

The simples approach?

- Scatter the data randomly across nodes
  - ✅ *Data is evenly distributed*
  - ✅ *Load is evenly distributed*

  What about searching for an item?
  - Query all the nodes for the data?
  - What can we do better?

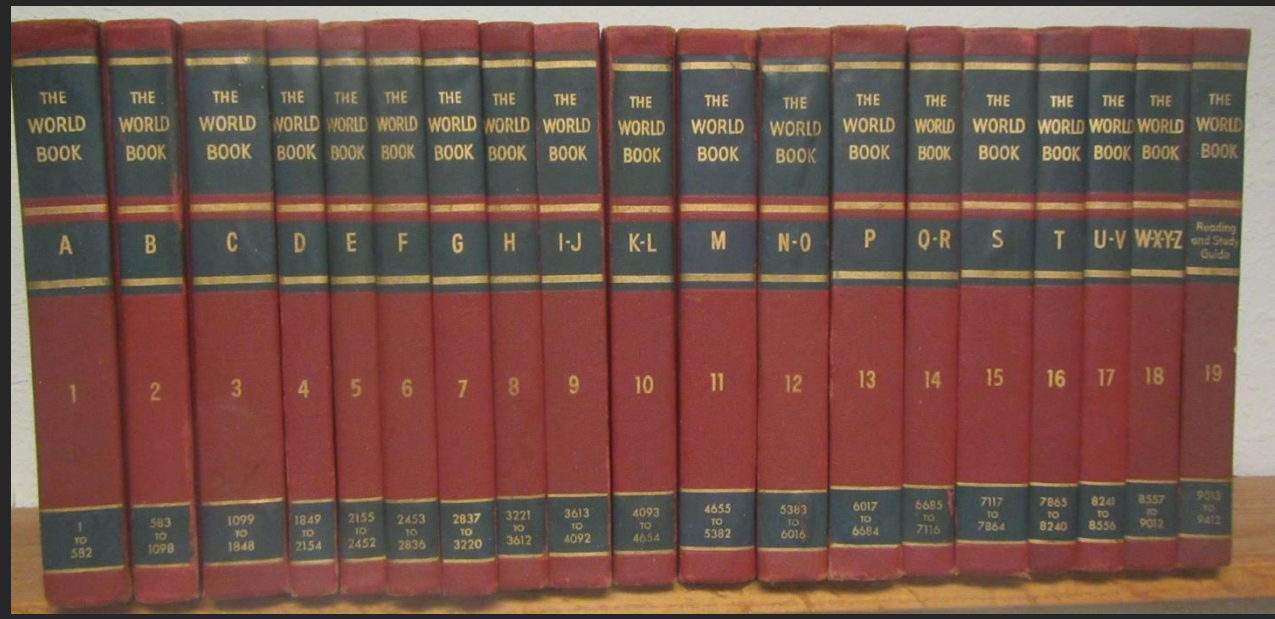
Partitioning by key range

- One way of partitioning is to assign a continuous range of keys (like a paper encyclopaedia)
- Range of keys not evenly spaced
- Sorted keys

- In some cases can lead to write hotspots

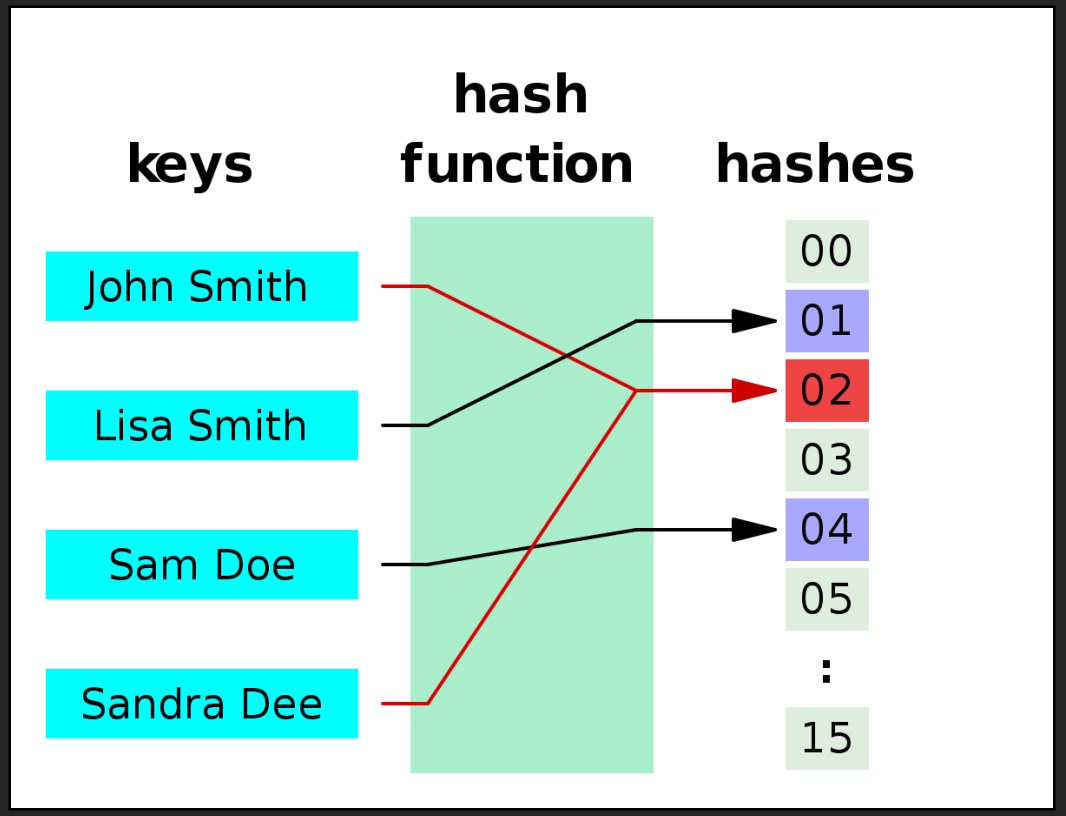
  If the key order follows the order the data arrives

  - (e.g., timestamp)



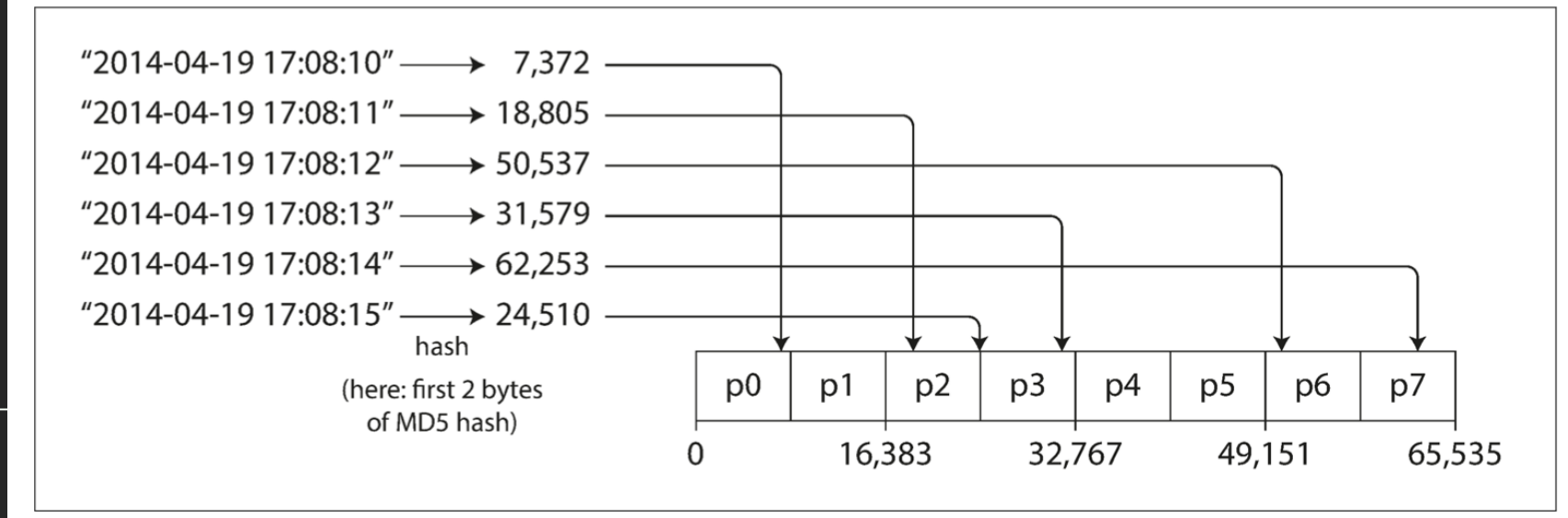
Partitioning by hash of key

- Uses a hash function to determine the partition for a given key.
- Assign to each partition a range of hashes (instead of a range of keys)



Partitioning by hash of key

- Uses a hash function to determine the partition for a given key.
- Assign to each partition a range of hashes (instead of a range of keys)



Partitioning by hash of key

- Good at distributing the keys fairly among the partitions
- Lose the ability to perform efficient range queries

  Keys that were once adjacent are now scattered across partitions

  - *Send the query to all partitions (MongoDB)*
  - *Use compound keys and hash the first one (Cassandra)*

Partitioning and secondary indexes

- Unlike the key in key-value models secondary indexes don’t uniquely identify a record

  Used for search/sorting purposes

  - E.g., the city of a social media user
- Secondary indexes don’t map neatly to partitions

Secondary indexes - Partitioning by document

Each document (record) lives in exactly one partition, determined by its primary key.

- Secondary indexes are **local** to the partition.
- Pros:

  Simple and Scalable

  Local operations are fast
- Cons:

  Queries on secondary attributes require contacting every partition

```
Partition 1: { user_id = 1, user_id = 4 }
Partition 2: { user_id = 2, user_id = 3 }

Each partition maintains:
   - A local secondary index on e.g. 'city'
```

Secondary indexes - Partitioning by term

- Instead of partitioning by document, the index itself is distributed by the secondary field (the "term").

  For example, you can have an inverted index mapping each city → list of user_ids.

```
Partition 1: cities starting with A–L
Partition 2: cities starting with M–Z
```

- Pros:

  Efficient queries on secondary fields.

  You can directly route the query to relevant partitions.

- Cons:

  More complex to maintain — writes must update both the document's partition and the index's partition.

Secondary indexes

Strategy	Partition Basis	Query Efficiency	Write Complexity	Used By
By Document	Primary key	Fast for key lookups, slow for secondary	Simple	MongoDB (local indexes)
By Term	Secondary key (term)	Fast for secondary queries	Complex, may require coordination	Cassandra, Elasticsearch

Rebalancing partitions

Rebalancing partitions

- Things change

  The query throughput increases,

  The dataset size increase

  A machine fails

- The process of moving data and load from one node in the cluster to another is called rebalancing

Rebalancing partitions

- After rebalancing, the load (data storage, read and write requests) should be shared fairly between the nodes in the cluster.
- While rebalancing is happening, the database should continue accepting reads and writes.
- No more data than necessary should be moved between nodes, to make rebalancing fast and to minimize the network and disk I/O load.

  Avoid moving data between partitions

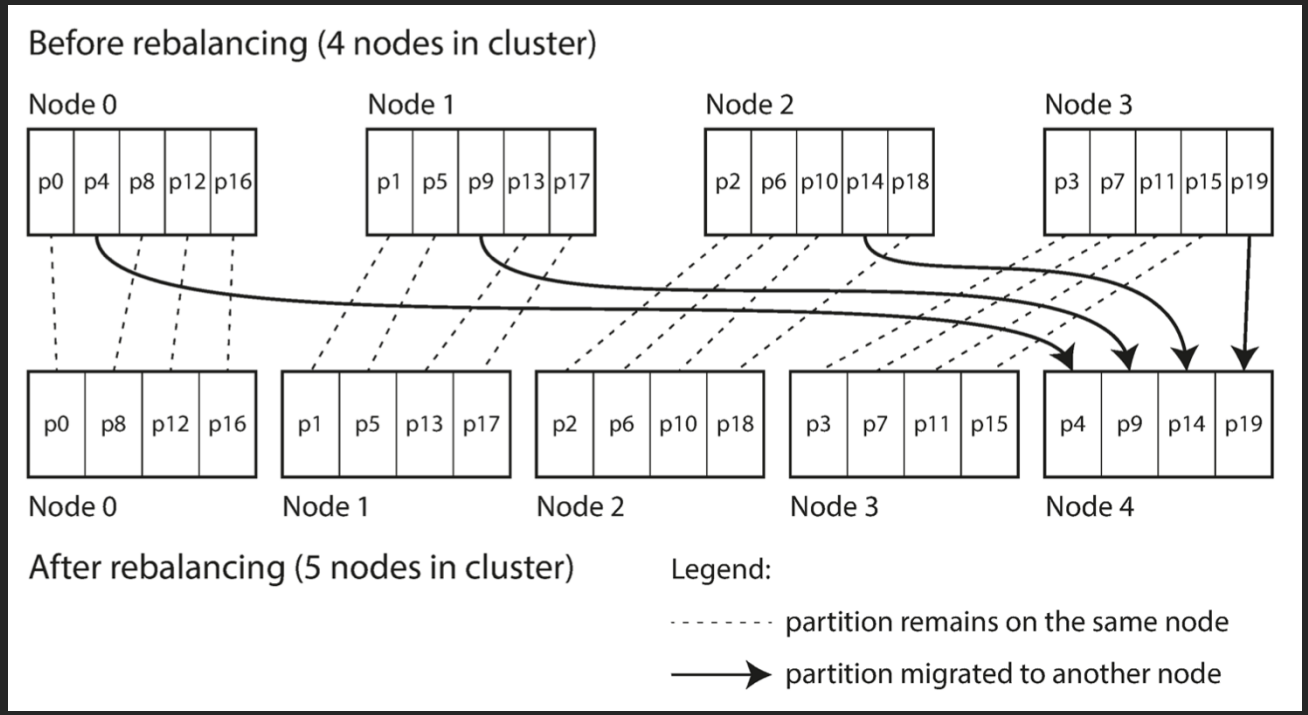
Fixed number of partitions

- I can create many more partitions than nodes

  e.g., split the data into 1,000 partitions to be stored in a 10 nodes cluster

  each node handling 100 partitions

- If nodes are added, they can steal partitions from other nodes
- Only entire partitions are moved between nodes.
- The number of partitions does not change, nor does the assignment of keys to partitions.



Dynamic partitioning

- If partitions grow beyond a certain size (e.g., 10GB) they are split into two

  Each part can be stored in a different node

- If the data becomes smaller, partitions can merge